CGL-sheet07

Robin Kühnberger[1] and Felix Naraschewski[2]

[1]University Stuttgart
[2]University Stuttgart

December 4, 2025

Contents

1 Aufgabe 1 **2**
1.1 Bib referenzen 3

2 List Comprehension **3**

3 Dateien lesen und Daten plotten **5**

1 Aufgabe 1

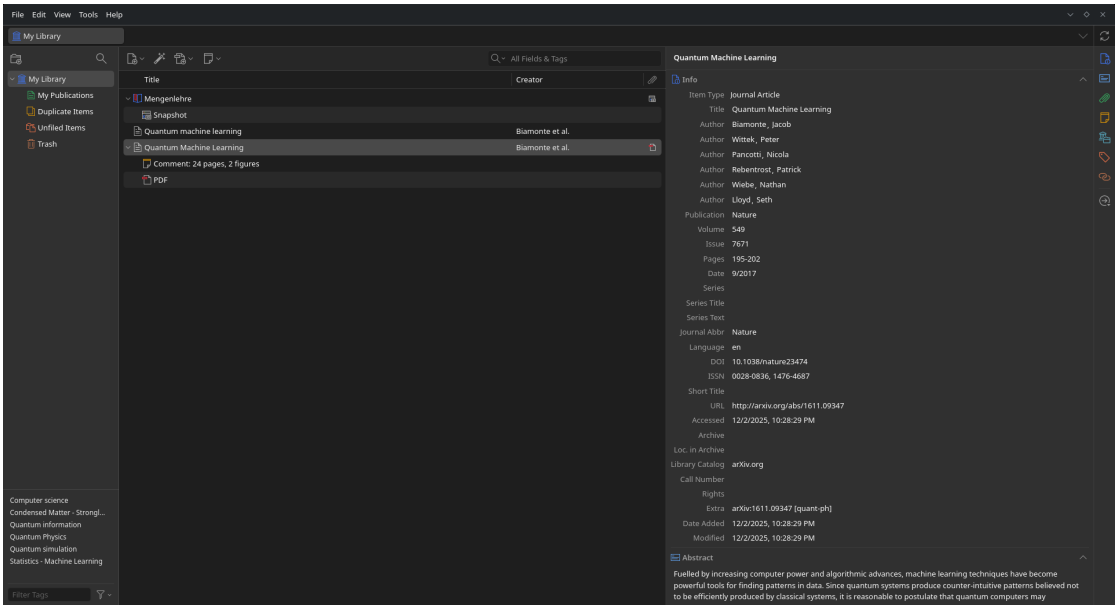


Figure 1: Artikel mit Metadaten

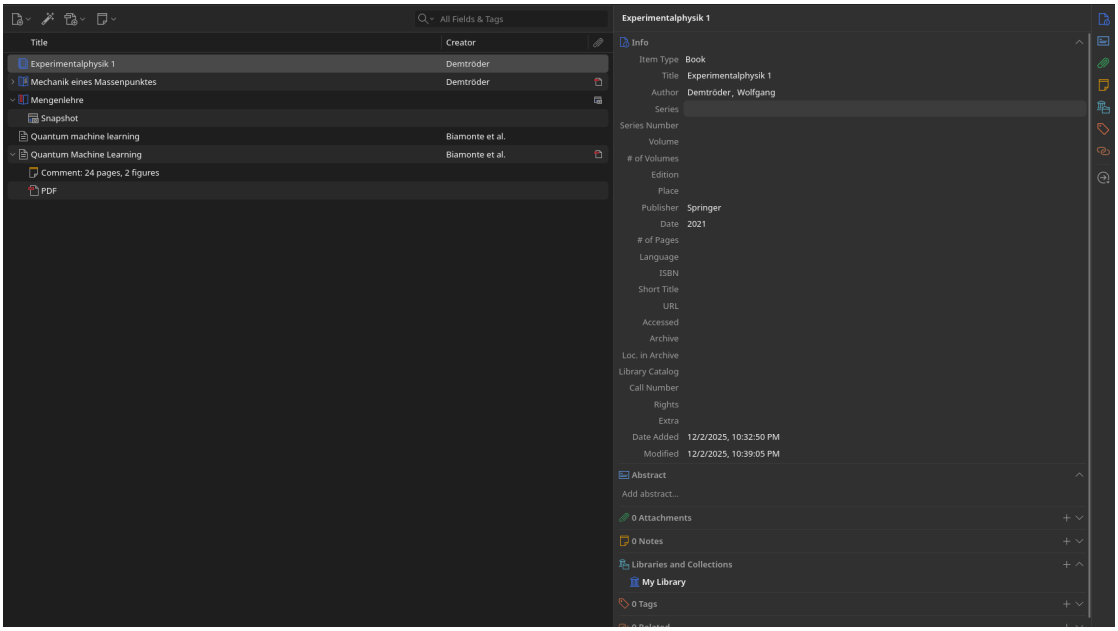


Figure 2: Experimentalphysik 1 Buch

1.1 Bib referenzen

[1] [2]

2 List Comprehension

```
import random
import matplotlib.pyplot as plt
import numpy as np


def generate_random_point_in_square(length):
    """
    Generate a random point in a square of a given side length centered
    ↪  around the origin with the sides aligned with x- and y-axis.
    """
    return tuple([length*random.uniform(-0.5, 0.5),
    ↪  length*random.uniform(-0.5, 0.5)])


def distance(p1, p2):
    """
    Measures the Euclidean distance between the points p1 and p2 in 2D.
    """
    a = np.array(p1)
    b = np.array(p2)
    d = np.linalg.norm(b-a)
    return d


def point_in_circle(p, center=None, radius=None):
    """
    Checks if the provided point p lies in a circle of a given radius
    ↪  around the provided center.
    """
    p = np.array(p)
    center = np.array(center)
    d = np.linalg.norm(p - center)
    if d < radius:
        return True
    else:
        return False
```

```
# TODO: generate a list of 1000 random points in a square of side length
↪  2.0 using generate_random_point_in_square and list comprehension
N = 1000
full_list_of_points = [generate_random_point_in_square(2) for _ in
↪  range(N)]

# TODO: generate a list of those points in full_list_of_points that are
↪  within a radius of 1.0 of the origin (again using list
↪  comprehension)
filtered_list_of_points = [
    point for point in full_list_of_points if point_in_circle(point, (0,
    ↪  0), 1.0)]

# TODO: print the ratio of the number of points in
↪  filtered_list_of_points to the total number of generated points
N_filtered = len(filtered_list_of_points)
ratio = N_filtered/N
print("ratio", ratio)

# TODO: Generate separate lists of the x- and y-values of
↪  full_list_of_points and filtered_list_of_points
x_values_full = np.array(full_list_of_points)[:, 0]
y_values_full = np.array(full_list_of_points)[:, 1]

x_values_filtered = np.array(filtered_list_of_points)[:, 0]
y_values_filtered = np.array(filtered_list_of_points)[:, 1]

plt.scatter(x_values_full, y_values_full)
plt.scatter(x_values_filtered, y_values_filtered)
plt.axis('equal')
plt.savefig('pi_montecarlo_plot.png')
plt.show()
```

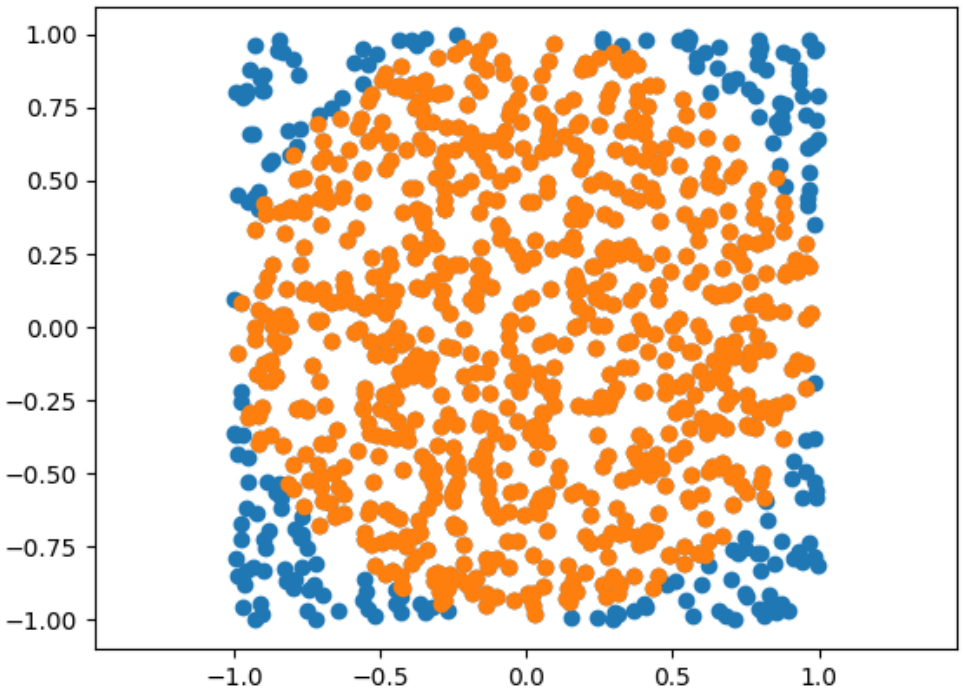


Figure 3: Scatter Plot

$$\begin{aligned} \text{Verhätnis} = \frac{N_{\text{filtered}}}{N} &\approx \frac{A_{\text{Kreis}}}{A_{\text{Quadrat}}} \\ \frac{(1)^2 \cdot \pi}{2^2} &= \frac{\pi}{4} \end{aligned} \tag{1}$$

3 Dateien lesen und Daten plotten

```
import matplotlib.pyplot as plt

dateiname = "messdaten.txt"
x_werte = []
y_werte = []

# 1. Datei öffnen und Schleife über Zeilen
with open(dateiname, 'r') as datei:
    print(f"Verarbeite Daten aus '{dateiname}'...")
```

```
    for zeilennummer, zeile in enumerate(datei, 1):

        # 2. .strip() verwenden
        bereinigte_zeile = zeile.strip()

        # 3. Filtern von leeren Zeilen und Kommentaren
        if not bereinigte_zeile or bereinigte_zeile.startswith('#'):
            # Ignoriere auch Zeilen, die mit '#' (Kommentar) beginnen
            continue

        # 4. Sonst Zeile ausgeben und Werte extrahieren
        print(f"Verarbeite: '{bereinigte_zeile}'")

        #Teile die bereinigte_zeile in Einzelteile auf
        teile = bereinigte_zeile.split()

        if len(teile) == 2:
            # Wandel die Teile in Gleitkommazahlen um
            x = float(teile[0])
            y = float(teile[1])

            # Speicher die Werte in den Listen x_werte und y_werte
            x_werte.append(x)
            y_werte.append(y)

# 5. Am Ende alle Tupel x-y plotten
if x_werte:
    print(f"Erfolgreich {len(x_werte)} Datenpunkte gesammelt. Erstelle
    ↪  Plot.")

    plt.figure()
    # Verwende Sie plt.scatter() um x_werte gegen y_werte zu plotten
    plt.scatter(x_werte,y_werte)

    plt.title('Messdaten aus Datei')
    plt.xlabel('X-Koordinate')
    plt.ylabel('Y-Koordinate')
    plt.grid(True)
    # plt.savefig('messdaten_plot.pdf')
    plt.savefig('messdaten_plot.png')

    plt.show()
else:
    print("Keine gültigen Daten zum Plotten gefunden.")
```

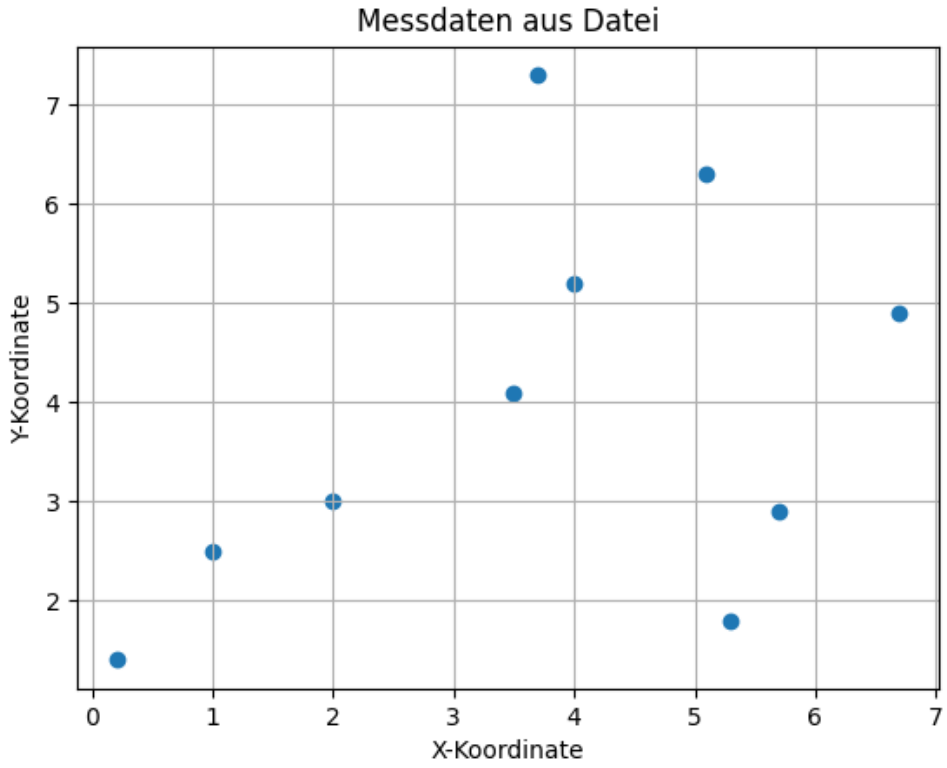


Figure 4: Scatter Plot

Quellen

References

[1] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum Machine Learning. *Nature*, 549(7671):195–202, September 2017. arXiv:1611.09347 [quant-ph].

[2] Wolfgang Demtröder. *Experimentalphysik 1.* Springer, 2021.

List of Figures

1 Artikel mit Metadaten 2

2 Experimentalphysik 1 Buch 2
3 Scatter Plot 5
4 Scatter Plot 7